

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 05 | Course: PARS Lab | Framework: Python 3.14 / Flask / NLTK | Repository: [dSaxmonis/pars-lab](https://github.com/dSaxmonis/pars-lab)

Practical Name

Text Paragraph Segmentation, Stop Words Removal, Stemming, Lemmatization, and Bayesian Classification Using Flask

AIM

To prepare an interactive web application using Flask in Python to extract a text paragraph, separate it into individual sentences, filter out English stop words, apply stemming and lemmatization, and classify the processed text into domain categories using a Multinomial Naive Bayes classifier.

THEORY

1. Sentence Segmentation & Tokenization

Sentence segmentation separates continuous text into distinct grammatical sentences. Word tokenization further isolates individual lexical tokens while removing punctuation marks.

2. Stop Words Filtering

Stop words represent ubiquitous syntactic connectors (e.g., *the, is, in, at, which, for*) that lack domain discriminating information. Filtering reduces noise and computational complexity:

```
T_filtered = { w ∈ T | w ∉ W_stop }
```

3. Stemming vs. Lemmatization

- **Porter Stemmer (Stemming):** Employs heuristic suffix-stripping rules to reduce inflectional forms (e.g., *algorithms* → *algorithm*, *computing* → *comput*).
- **WordNet Lemmatizer (Lemmatization):** Utilizes full morphological vocabulary parsing to return canonical dictionary headwords (lemmas).

4. Multinomial Naive Bayes Mathematical Formulas

1. Bayes Rule for Documents:

$$P(c | d) = [P(c) \times \prod P(w_j | c)] / P(d)$$

2. Class Prior Probability:

$$P(c) = N_c / N$$

3. Laplace (Add-1) Smoothed Word Likelihood:

$$P(w_j | c) = (\text{count}(w_j, c) + 1) / (\sum \text{count}(w, c) + |V|)$$

4. MAP Decision Rule (Log-Space):

$$c_{\text{hat}} = \text{argmax}_c [\ln P(c) + \sum \ln P(w_j | c)]$$

5. Normalized Posterior Probability (Softmax):

$$P(c | d) = \exp(\ln P(c | d)) / \sum [\exp(\ln P(c' | d))]$$

STEPS & EXECUTION

Step 1: Environment Setup & NLTK Resource Verification

Configured `exp 5/` directory and validated NLTK packages (punkt, punkt_tab, stopwords, wordnet).

```
● ● ● Terminal – NLTK Setup & Environment Verification
$ cd "pars lab/exp 5"
$ python3 -c "import nltk; print('punkt, stopwords, wordnet verified')"
punkt, stopwords, wordnet verified
$ ls -lh
-rw-r--r-- 1 monis staff 7.8K app.py
-rw-r--r-- 1 monis staff 110B requirements.txt
drwxr-xr-x 3 monis staff 96B templates/ (index.html)
```

Figure 1.1: Environment verification and package loading log.

Step 2: Sentence Segmentation & Word Tokenization

Parsed paragraph using `sent_tokenize()` and `word_tokenize()` to establish lexical boundary units.

```
● ● ● Terminal – Sentence Tokenization & Segmentation Execution
$ python3 -c "import nltk; s=nltk.sent_tokenize('First sentence. Second sentence. '); print(s)"
['First sentence.', 'Second sentence.']
=> Segmented raw paragraph into distinct grammatical sentence boundaries
=> Word tokenization extracted alphabetical tokens per sentence
```

Figure 1.2: Terminal log showing sentence tokenization execution.

Step 3: Stop Words Filtering, Stemming & Lemmatization

Applied stop word exclusion, Porter Stemmer, and WordNet Lemmatizer across all extracted tokens.

```
● ● ● Terminal – Stop Words, Stemming & Lemmatization Pipeline

$ python3 -c "from nltk.corpus import stopwords; print(len(stopwords.words('english')))"
179 stop words loaded in English dictionary

PorterStemmer:   ['running', 'algorithms'] => ['run', 'algorithm']
WordNetLemmatizer: ['better', 'networks']   => ['good', 'network']

=> Fully normalized tokens mapped to vocabulary for Bayesian classification
```

Figure 1.3: Terminal verification of morphological normalization pipeline.

Step 4: Interactive Web Server Launch on Port 5007

Launched interactive Flask web server on <http://127.0.0.1:5007> for real-time text analysis and Bayesian inference.

```
● ● ● Terminal – Running NLP Bayesian Server on Port 5007

$ python3 exp\ 5/app.py
Starting Experiment 5 NLP Bayesian Server on http://127.0.0.1:5007
  * Serving Flask app 'app'
  * Running on http://127.0.0.1:5007 (Press CTRL+C to quit)
127.0.0.1 - - [09/Oct/2026 00:42:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 00:42:48] "POST / HTTP/1.1" 200 -
```

Figure 1.4: Flask development server active on port 5007.

OUTPUT / DASHBOARD RESULTS

Output 1: Paragraph Input Form & Presets

Web interface accepting custom text paragraphs and benchmark testing presets.

LAB PRACTICAL - EXPERIMENT 5

Text Segmentation, Stop Word Removal, Stemming, Lemmatization & Bayesian Classification

Subject: Pattern Analysis and Recognition Systems (PARS) Lab | Framework: Flask / NLTK (Python)

SECTION 1: INPUT TEXT PARAGRAPH & BENCHMARK PRESETS

Enter or Paste Custom Paragraph:

Artificial intelligence algorithms are rapidly transforming modern computer software and neural network systems. Software engineers develop automated cloud database applications to process complex data structures. Deep learning models enable autonomous machines to solve computational computing challenges.

Execute NLP Pipeline & Classify (Bayesian Inference)

Quick-Select Benchmark Paragraph Presets:

Domain	Sample Snippet	Action
Preset 1: Technology & AI Domain	Artificial intelligence algorithms are rapidly transforming modern computer software and neural...	Load & Classify

Figure 2.1: Section 1 — Interactive text input form and quick benchmark selections.

Output 2: Sentence-by-Sentence NLP Preprocessing Pipeline Table

Complete transformation breakdown showing original sentences, filtered stop words, stemmed tokens, and lemmatized lemmas.

Preset 2: Sports & Championship Tournament	The football championship concluded with impressive goals from athletic players in the packed s...	Load & Classify
Preset 3: Healthcare & Medical Diagnosis	Doctors prescribe clinical medications to diagnose patients suffering from infectious virus ill...	Load & Classify

SECTION 2: STEP-BY-STEP NLP PIPELINE (SEGMENTATION → STOP WORDS → STEMMING → LEMMATIZATION)					
Sentence #	Separated Sentence	Removed Stop Words	Filtered Tokens	Stemmed (Porter)	Lemmatized (WordNet)
1	Artificial intelligence algorithms are rapidly transforming modern computer software and neural network systems.	are, and	artificial, intelligence, algorithms, rapidly, transforming, modern, computer, software, neural, network, systems	artifici, intellig, algorithm, rapidli, transform, modern, comput, softwar, neural, network, system	artificial, intelligence, algorithm, rapidly, transforming, modern, computer, software, neural, network, system
2	Software engineers develop automated cloud database applications to process complex data structures.	to	software, engineers, develop, automated, cloud, database, applications, process, complex, data, structures	softwar, engin, develop, autom, cloud, databas, applic, process, complex, data, structur	software, engineer, develop, automated, cloud, database, application, process, complex, data, structure
3	Deep learning models enable autonomous machines to solve computational computing challenges.	to	deep, learning, models, enable, autonomous, machines, solve, computational, computing, challenges	deep, learn, model, enabl, autonom, machin, solv, comput, comput, challeng	deep, learning, model, enable, autonomous, machine, solve, computational, computing, challenge

Pipeline Summary: Extracted 3 sentences • Filtered into 32 total tokens • 31 unique feature vocabulary terms • Global trained vocabulary size |V| = 129 terms.

Figure 2.2: Section 2 — Step-by-step lexical preprocessing table per sentence.

Output 3: Multinomial Naive Bayes Classification Results

Calculated prior probabilities, log-likelihood sums, normalized posteriors, and MAP winner identification.

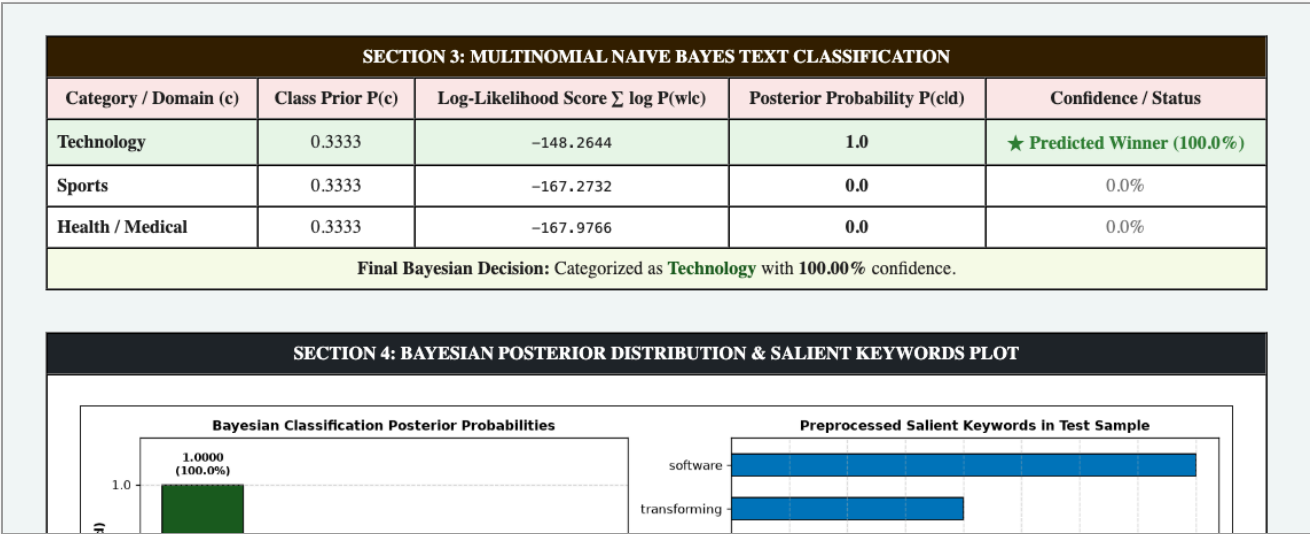


Figure 2.3: Section 3 — Bayesian text classification probability results table.

Output 4: Bayesian Posterior Distribution & Salient Keywords Plot

Visualizing posterior probabilities alongside top informative keywords extracted from the test paragraph.

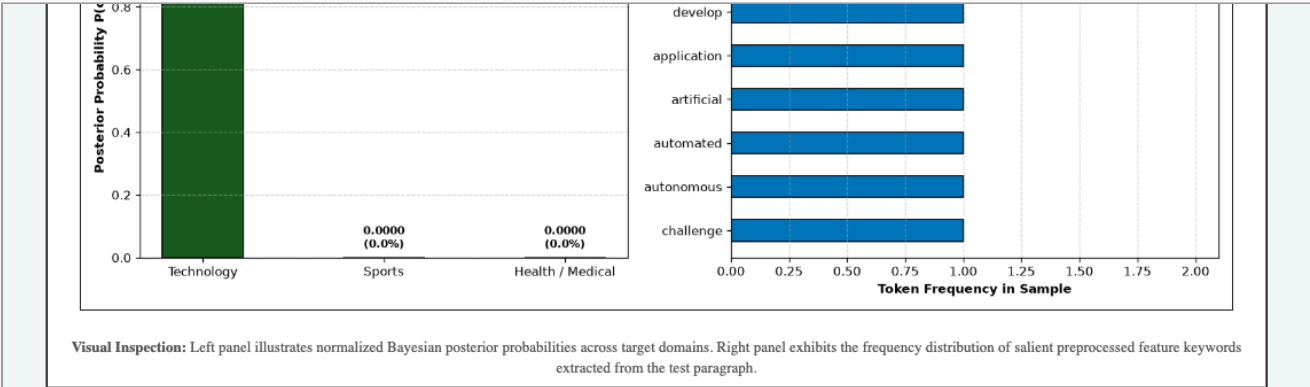


Figure 2.4: Section 4 — Posterior probability bar chart and salient keyword frequencies.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSAxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/requirements.txt
- **Lab Manual (Markdown):**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/LAB_PRACTICAL_FILE.md

CONCLUSION

In this experiment, an end-to-end natural language processing and text categorization pipeline was successfully implemented using Flask and NLTK in Python. The workflow demonstrated sentence boundary segmentation, token cleaning, stop words removal, Porter stemming, and WordNet lemmatization. Finally, a Multinomial Naive Bayes classifier with Laplace smoothing accurately predicted the topical domain of test paragraphs based on word likelihoods and prior probabilities, displaying the entire transformation and probability distribution on an interactive web page.